TEIL 3 · OBJEKTORIENTIERTE SOFTWAREENTWICKLUNG

Klassen in Python

Vom Entwurfsdiagramm zum Programm: Konstruktor, `self`, Objekte erzeugen und Referenzen

WarnowBank Rostock · Baustein 3 · ca. 15 min · GuiPy bereithalten

ÜBERSETZEN

Vom Entwurfsdiagramm zum Code

Kunde
kundennummer : int name : str vorname : str geburtsdatum : str
name_aendern daten_drucken

Antippen: Was gehört zusammen?

```
class Kunde:
    def __init__(self, kundennummer, name, vorname, geburtsdatum):
        self.kundennummer = kundennummer
        self.name = name
        self.vorname = vorname
        self.geburtsdatum = geburtsdatum
```

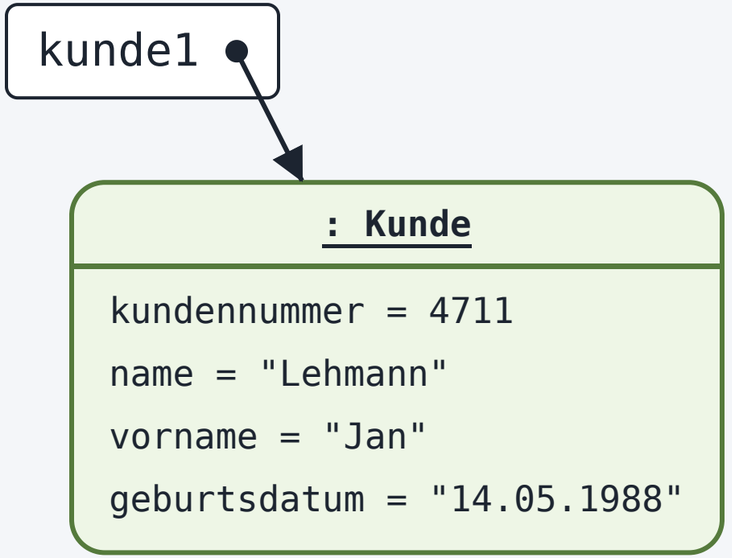
- `class Kunde:` – Klassenname groß, alles Eingerückte gehört dazu
- `__init__` – der **Konstruktor**: legt die Attribute an (je zwei Unterstriche)
- Die Datentypen stehen im Code nicht: Python erkennt sie am Wert – `4711` ist ein `int`
- Methoden folgen in Baustein 4

GENAU HINGESCHAUT

Was passiert beim Erzeugen eines Objekts?

```
class Kunde:
    def __init__(self, kundennummer, name, vorname, geburtsdatum):
        self.kundennummer = kundennummer
        self.name = name
        self.vorname = vorname
        self.geburtsdatum = geburtsdatum

kunde1 = Kunde(4711, "Lehmann", "Jan", "14.05.1988")
```



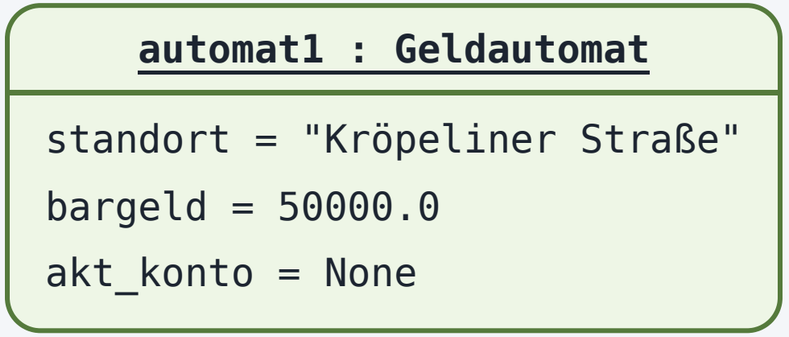
Der Konstruktor endet, die Parameter verschwinden. In `kunde1` wird eine **Referenz** auf das Objekt gespeichert – im Objektdiagramm: kunde1 : Kunde.

KONSTRUKTOR

Nicht jedes Attribut ist ein Parameter

```
class Geldautomat:
    def __init__(self, standort):
        self.standort = standort
        self.bargeld = 50000.0
        self.akt_konto = None

automat1 = Geldautomat("Kröpeliner Straße")
```



- Was für jedes Objekt **anders** ist, wird als Parameter übergeben
- Was für alle neuen Objekte **gleich** ist, setzt der Konstruktor selbst: **fester Startwert**
- `None`: Das Attribut verweist (noch) auf kein Objekt

IM INTERPRETER

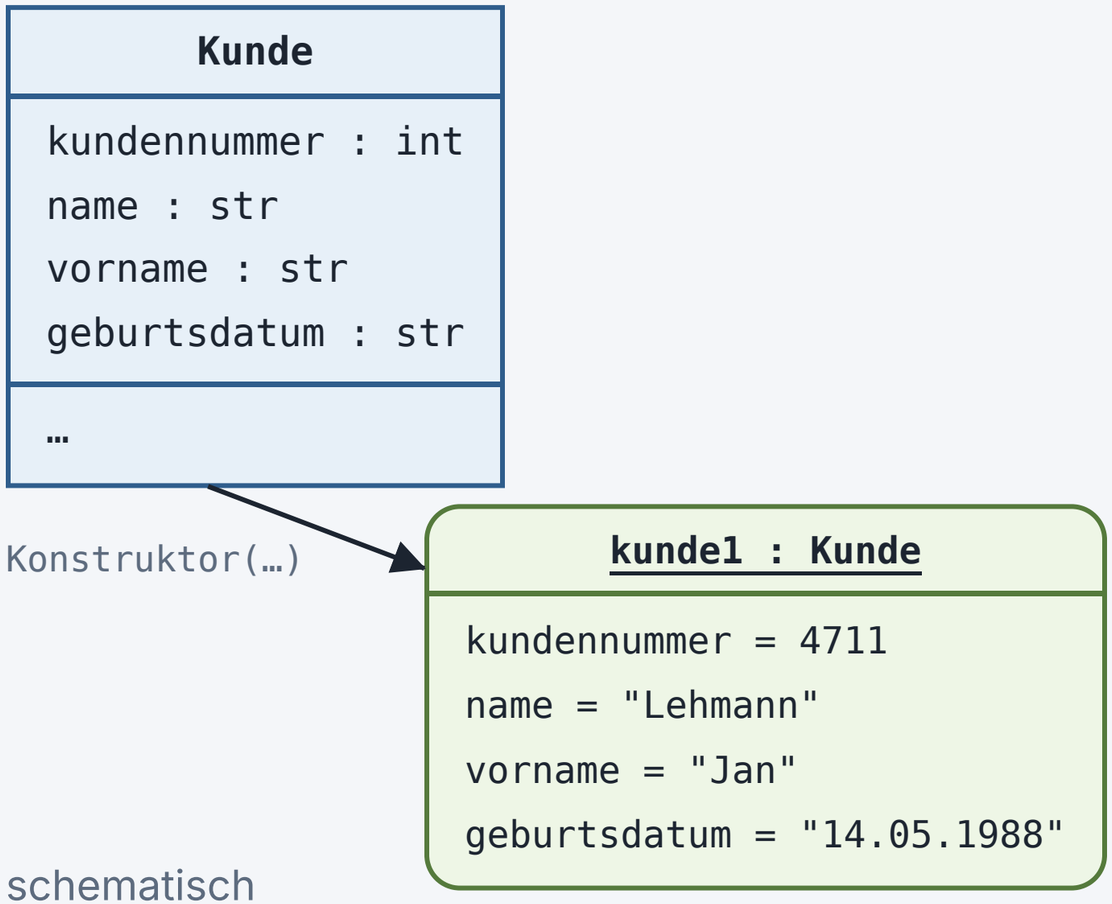
Objekte erzeugen und benutzen

```
# Datei ausführen, dann im Interpreter:
>>> kunde1 = Kunde(4711, "Lehmann", "Jan", "14.05.1988")
>>> kunde1.vorname
'Jan'
>>> kunde1.name = "Lehmann-Krüger"
>>> kunde1.name
'Lehmann-Krüger'
```

- **Instanziierung:** `objekt = Klasse(Werte)` – `self` wird nicht mitgeschrieben
- **Punktnotation:** `objekt.attribut`
- Zuweisung ändert den Zustand (ob das gut ist, klären wir in Baustein 5)

IN GUIPY

Objekte im UML-Fenster



schematisch

1. Datei speichern und ausführen
2. *Diagramm aus geöffneten Dateien*: Die Klassen erscheinen als Entwurfsdiagramm
3. Rechtsklick auf die Klasse → **Konstruktor**, Werte eingeben → das Objekt erscheint mit seinen Attributwerten
4. Rechtsklick auf das Objekt: Attribute bearbeiten, Methoden aufrufen – danach wird der neue Zustand angezeigt

Code geändert? Die Klasse wird **violett** → neu ausführen. Die Objekte gehen dabei verloren.

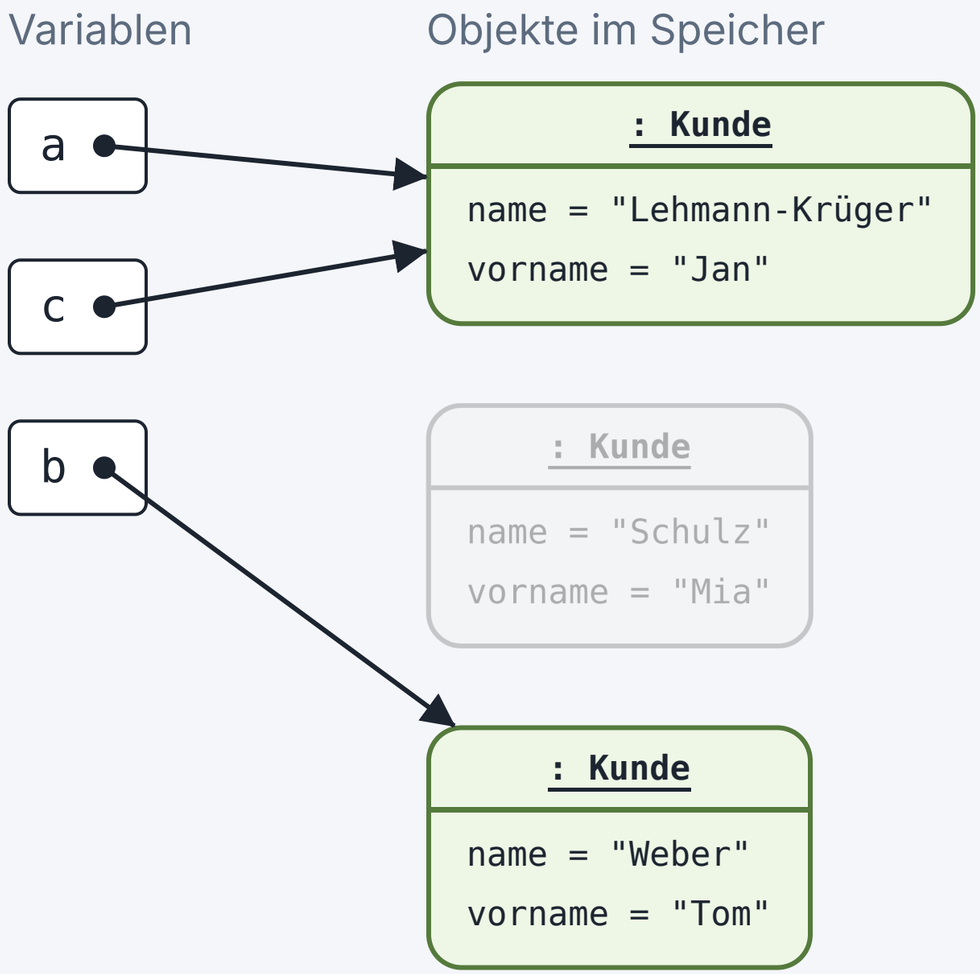
GENAU HINGESCHAUT

Variablen speichern Referenzen

```
a = Kunde(4711, "Lehmann", "Jan", "14.05.1988")
b = Kunde(4712, "Schulz", "Mia", "02.11.2001")
c = a
c.name = "Lehmann-Krüger"
b = Kunde(4713, "Weber", "Tom", "30.01.1999")
```

b verweist auf ein neues Objekt. Auf Frau Schulz zeigt nichts mehr – das Objekt ist nicht mehr erreichbar und wird von Python gelöscht.

Schritt 6 / 6



WENN ES NICHT LÄUFT

Fehlermeldungen lesen

`NameError: name 'Kunde' is not defined`

Die Datei wurde nicht ausgeführt – oder der Klassenname ist falsch geschrieben (`kunde` statt `Kunde`).

`IndentationError: expected an indented block after function definition`

Der Rumpf von `__init__` ist nicht eingerückt. Alles, was zum Konstruktor gehört, steht eine Stufe weiter rechts.

`TypeError: Kunde.__init__() takes 5 positional arguments but 6 were given`

Beim Erzeugen wurde ein Wert zu viel übergeben. Python zählt `self` mit: 4 Werte + `self` = 5.

`AttributeError: 'Kunde' object has no attribute 'nachname'`

Das Attribut heißt anders: Im Konstruktor wurde `self.name` angelegt, nicht `self.nachname`.

Faustregel: Die letzte Zeile der Meldung wörtlich übersetzen – sie verrät fast immer die Ursache.

INS GLOSSAR

Das Wichtigste aus Teil 3

Konstruktor	Methode `__init__`, die beim Erzeugen automatisch ausgeführt wird; legt die Attribute an und belegt sie mit Parametern oder festen Startwerten
`self`	das Objekt, das gerade erzeugt bzw. benutzt wird; immer erster Parameter
Instanziierung	Erzeugen eines Objekts: `objektname = Klassenname(Werte)`
Referenz	Variablen speichern Verweise auf Objekte; `c = a` kopiert den Verweis, nicht das Objekt

JETZT SIE

Etappe 3.B: Klassen in Python

- Aufgabe 3.6 – Kunde und Konto implementieren (in GuiPy, neue Datei `bank.py`)
- Aufgabe 3.7 – Zwei gleiche Kunden?
- Aufgabe 3.8 – Wie viele Konten gibt es?
- Aufgabe 3.9 – Fehlersuche
- Wer fertig ist: 3.10 Die Klasse Karte, 3.11 Das Konto kennt seinen Inhaber (LK verbindlich)

ca. 40 min (LK 55 min) · danach Stopp

SICHERUNG 3.B

Unser Stand von `bank.py`

```python
class Kunde:
    def __init__(self, kundennummer, name, vorname, geburtsdatum):
        self.kundennummer = kundennummer
        self.name = name
        self.vorname = vorname
        self.geburtsdatum = geburtsdatum

class Konto:
    def __init__(self, kontonummer, pin):
        self.kontonummer = kontonummer
        self.pin = pin
        self.kontostand = 0.0
        self.dispo = 500.0
```

Danach erhalten alle diesen Stand über den Austauschordner.

Kurz-Check

Unterschied: `Konto`, `konto1`, `konto1.kontostand`

Klasse (Bauplan) – Objekt (Variable mit Referenz) – Attribut des Objekts

Aufgabe des Konstruktors

legt beim Erzeugen die Attribute an und belegt sie mit Startwerten

„falsch“ in Python

`False` – großgeschrieben, ohne Anführungszeichen